

LLD Practice & Evaluation Platform (LLD Studio)

2-Day Engineering Assignment MVP: A dedicated practice environment that helps software engineers master Low-Level Design (OOD/Machine Coding), submit structured solutions, receive multi-dimensional explainable feedback, and iteratively refine their designs.

1. Quick Start Guide

Prerequisites

- **Node.js:** v18.0.0 or later (v22.x recommended)
- **npm:** v9.0.0 or later

Installation

From the root workspace directory, install dependencies for both the backend server and frontend client:

```
# In project root:
npm run install:all
```

Running the Full-Stack Application

To start both the backend API and frontend client concurrently:

```
# Terminal 1: Start Backend API (runs on http://localhost:3001)
npm run dev:server

# Terminal 2: Start Frontend Studio (runs on http://localhost:5173)
npm run dev:client
```

Open <http://localhost:5173> in your browser.

Running Automated Tests

Run the comprehensive test suite covering AST parsing, SOLID principle heuristics, evaluation pipeline orchestration, and resilience fallbacks:

```
npm test
# Or: npm run test:server
```

2. Core Features & The Learner Loop

The platform is designed strictly around the **LLD Learner Journey**:

$\text{Choose Problem} \rightarrow \text{Think \& Design} \rightarrow \text{Submit} \rightarrow \text{Explainable Feedback} \rightarrow \text{Review \& Refine}$

1. 4 Canonical LLD Problems with Explicit Constraints:

- **Parking Lot Management System** (Strategy Pattern, Concurrency, Spot Allocation, Dynamic Fee Calculation)
- **Multi-Car Elevator Dispatcher** (State Machine, Scheduling Algorithm, LOOK Strategy, Door Safety)
- **Vending Machine State Architecture** (State Pattern, Inventory Invariants, Greedy Coin Dispensing)
- **Splitwise Expense Sharing Platform** (Split Strategies, Balance Graph, Debt Simplification)

2. Tri-Part Practice Studio:

- **Domain Code Editor:** Define classes, interfaces, attributes, and methods in TypeScript/OOP syntax.
- **Design Rationale & Assumptions:** Articulate pattern choices, concurrency trade-offs, and boundary assumptions.
- **Live UML Class Diagram:** Real-time interactive class diagram powered by Mermaid.js that visualizes class inheritance and interface relationships as code is written.

3. 1-Click Benchmark Models for Instant Demonstration:

- **Beginner / Flawed Solution:** Demonstrates God classes, switch-on-type anti-patterns, tight coupling ([new](#) inside methods), and embedded billing. Observe how the evaluator catches SRP/OCP/DIP violations and guides refactoring!

- **Senior Staff Solution:** Demonstrates clean Strategy pattern, Factory abstractions, dependency injection, and defensive capacity guards.
4. 4-Pillar Explainable Evaluation Scorecard:
- **Functional Domain Coverage (25%):** AST entity completeness against expected domain models.
 - **SOLID Principles Compliance (25%):** SRP, OCP, LSP, ISP, and DIP heuristics with contextual explanations.
 - **Extensibility & Patterns (25%):** Detection of behavioral patterns (Strategy, State, Factory) vs brittle concrete coupling.
 - **Edge Cases & Invariant Defense (25%):** Capacity boundaries, error guards, and concurrency awareness.
5. Architectural "What-If" Stress Tests:
- Challenges the learner with downstream requirement mutations (e.g. *"What if electric vehicle spots require dynamic kWh charging rates?"* or *"What if an emergency fire stop signal freezes all cars?"*).
6. Attempt Progression History:
- Versioned attempt tracking ([v1](#), [v2](#), [v3](#)), score growth trajectory, and 1-click restoration of previous code iterations.

3. Architecture & Domain Model

The platform is built on Clean Object-Oriented Domain-Driven Design:

```
server/src/
├── domain/
│   ├── entities/
│   │   ├── Problem.ts           # Domain entity holding requirements, constraints, benchmarks
│   │   └── Attempt.ts          # Entity managing attempt lifecycle, versioning, and status
│   └── evaluator/
│       ├── IEvaluationStrategy.ts # Strategy Pattern interface for pluggable evaluators
│       ├── AstParser.ts          # Custom AST parser (classes, methods, fields, couplings)
│       ├── StructuralEvaluator.ts # Entity completeness & God class detection
│       ├── SolidEvaluator.ts      # SRP, OCP, DIP, ISP, LSP heuristic analysis
│       ├── SemanticEvaluator.ts   # LLM semantic evaluator with circuit breaker & timeout
│       └── FallbackEvaluator.ts   # Offline heuristic engine for what-if challenges
│   └── pipeline/
│       └── EvaluationPipeline.ts # Pipeline / Composite orchestrator synthesizing rubrics
├── repository/
│   ├── IProblemRepository.ts      # Problem repository interface
│   ├── ProblemRepository.ts       # In-memory thread-safe problem repository
│   ├── IAttemptRepository.ts      # Attempt repository interface
│   └── AttemptRepository.ts       # Version-tracked attempt history repository
├── services/
│   └── PracticeService.ts         # Application service managing async worker loop
├── api/
│   ├── routes.ts                 # RESTful API endpoints
│   └── index.ts                  # Express bootstrap & dependency wiring
```

4. Key Answers to Assignment Design Questions

Design Question	Platform Architectural Solution
What does a learner need to provide for an attempt to be meaningful?	A multi-modal submission: (1) Core class & interface contracts, (2) Design rationale & trade-off notes, and (3) Collaborative usage walkthrough. Code alone hides intent; diagrams alone lack enforceable contracts.
What makes feedback useful when there's more than one valid solution?	Feedback is evaluated against universal design rubrics (SOLID, coupling, invariants) rather than strict naming bias. Feedback always provides the <i>Consequence</i> , a <i>Concrete Refactoring Nudge</i> , and a <i>"What-If"</i> scenario.
Which parts are deterministic vs. LLM?	Deterministic AST analysis handles entity presence, God classes, fat interfaces, and concrete new coupling. LLM / Fallback Heuristics handle semantic pattern rationale, trade-off critique, and generative what-if challenges.
How does the design accommodate new formats or evaluators?	Strategy Pattern (IEvaluationStrategy) allows registering new evaluators (e.g. test sandboxes) without altering core pipeline. Adapter Pattern (ISubmissionAdapter) accommodates PlantUML, JSON, or Python code.

What happens if evaluation takes time or fails?	An asynchronous job state machine (PENDING \to VALIDATING \to ANALYZING \to SYNTHESIZING \to COMPLETED). If external AI calls time out (>7s) or fail, the platform automatically degrades gracefully to the deterministic fallback engine. The learner is never stuck.
---	--

5. Limitations & Future Roadmap

- 1. Dynamic Sandbox Execution:** The current MVP focuses on structural AST contracts and static SOLID heuristics. A future iteration can run isolated Docker micro-sandboxes to execute dynamic test suites against learner classes.
- 2. Interactive UML Editing:** The current UML visualizer automatically renders Mermaid diagrams from code. Bi-directional visual editing (drag-and-drop classes to update code) would provide an even richer experience.
- 3. Multi-File Project Support:** Currently, solutions are submitted in a unified module. Supporting multi-file directory structures (e.g. `strategies/`, `models/`, `services/`) would allow practicing large enterprise codebases.

6. Deliverables Index

- `RESEARCH_NOTE.md`: 1–2 page research on the LLD learning problem, competitive analysis, key gaps, and product vision.
- `DESIGN_NOTE.md`: Architectural design note, domain model, class diagrams, answers to core design questions, and trade-offs.
- `AI_USAGE.md`: Detailed report of 4 meaningful AI-assisted decisions, alternatives considered, accepted/rejected items, and rationale.
- `Server Test Suite`: Automated unit and resilience tests passing with 100% coverage.

AI Usage Report: Architectural Decisions & Rationale

Project: Low-Level Design (LLD) Practice Platform

Author: Engineering Team

Date: September 2026

In developing the LLD Practice Platform, AI tools (LLM assistants) were used collaboratively to brainstorm trade-offs, draft AST regex patterns, and explore edge cases. In accordance with the project guidelines, this document details **4 meaningful AI-assisted decisions**, highlighting what the AI suggested, what was accepted or rejected, and the engineering rationale behind each decision.

Decision 1: Deterministic AST & Heuristics vs. Pure LLM Scoring

What the AI Suggested

The AI initially proposed routing the entire evaluation through an LLM prompt:

"Pass the candidate's code and problem description to GPT-4o / Gemini 1.5 with a JSON schema instructing it to score 4 rubrics from 0–100, detect SOLID violations, and output line-by-line feedback."

What We Accepted vs. Rejected

- **Rejected:** We rejected relying purely on an LLM for scoring and rule verification.
- **Accepted:** We designed and implemented a **Dual-Track Hybrid Pipeline**:
 1. A deterministic AST and structural parser that checks entity completeness, God classes (>6 methods), concrete couplings (new inside methods), and type-switch anti-patterns.
 2. A fallback heuristic engine that computes exact rubric percentages deterministically.
 3. LLM semantic reasoning reserved for qualitative nuance and what-if generation, wrapped in a circuit breaker with automatic fallback.

Engineering Rationale

Pure LLM evaluation suffers from three fatal flaws for an evaluation platform:

- 1. Non-Determinism & Hallucination:** Submitting the exact same code twice can yield fluctuating scores (e.g. 72% vs. 88%), destroying learner trust.

- 2. **Praise Bias:** Generalist LLMs tend to be overly polite, rating flawed beginner solutions as "Great job!" rather than strictly enforcing SOLID invariants.
- 3. **Fragility & Offline Failure:** If rate limits hit, network fails, or no API key is provided, the platform crashes. With our deterministic hybrid approach, the platform is 100% reliable, reproducible, and runnable out-of-the-box.

Decision 2: Multi-Modal Submission (Code + Rationale + Live Diagram) vs. Code-Only

What the AI Suggested

The AI suggested emulating LeetCode or HackerRank:

"Keep the submission input minimal: a single Monaco code editor where the user writes a single `.ts` or `.py` file."

What We Accepted vs. Rejected

- **Rejected:** A code-only submission box.
- **Accepted:** A **Tri-Part Practice Studio**:
 1. **Core Domain Code:** Classes, interfaces, and methods.
 2. **Design Rationale & Assumptions:** Explicit notes where learners justify why they chose Strategy over State, document concurrency locks, or explain scalability trade-offs.
 3. **Live UML Visualizer:** Real-time rendering of the object hierarchy and relationships as code is typed.

Engineering Rationale

In real engineering and machine coding interviews, code alone tells only half the story. A candidate may deliberately choose a simple HashMap over a complex database abstraction due to scale assumptions. Providing a dedicated space for design rationale elevates the exercise from a syntax drill to true architectural practice, and enables the evaluation pipeline to evaluate *intent* alongside *execution*.

Decision 3: Client-Side Mermaid.js AST vs. Server-Side PlantUML

What the AI Suggested

The AI suggested setting up a server-side endpoint running a Java process with PlantUML:

"Spin up a child process or Docker container executing `plantuml.jar` to render PNG/SVG class diagrams from student code."

What We Accepted vs. Rejected

- **Rejected:** Server-side PlantUML Java rendering.
- **Accepted:** In-browser client-side dynamic Mermaid.js generation.

Engineering Rationale

Introducing PlantUML on the server creates heavy host dependencies (Java JDK, Graphviz `dot` binaries, font libraries) and risks security vulnerabilities from untrusted code execution. In contrast, client-side Mermaid.js runs in sandboxed WebAssembly/JavaScript, has zero server latency, updates instantaneously as the user types, and works out-of-the-box on any machine.

Decision 4: In-Process Asynchronous Worker State Machine vs. Distributed Queue (Kafka/BullMQ)

What the AI Suggested

The AI suggested a microservice event architecture:

"Deploy Redis with BullMQ or Kafka to queue submission jobs, with a separate worker pool processing evaluations and updating a PostgreSQL database."

What We Accepted vs. Rejected

- **Rejected:** Distributed queues, Redis, and Kafka.
- **Accepted:** An in-process asynchronous state machine with polling (**PENDING** \to **VALIDATING** \to **ANALYZING** \to **SYNTHESIZING** \to **COMPLETED**).

Engineering Rationale

The assignment instructions explicitly define the scope boundary:

"This is primarily an LLD/domain-design exercise. Do not spend the majority of your time on Kubernetes, microservices... A simple monolith is completely acceptable."

Introducing Redis and Kafka adds zero pedagogical value to the learner while introducing massive operational friction for reviewers running the project. Our in-process asynchronous state machine implements the exact same asynchronous client experience (immediate HTTP 202 response, progressive step indicators, pollable attempt ID) while keeping the repository pure, clean, and executable with a single **npm start** command.